

UNIVERSIDADE DA CORUÑA

Programación de Sistemas

2.3 Fragments and Dialogs

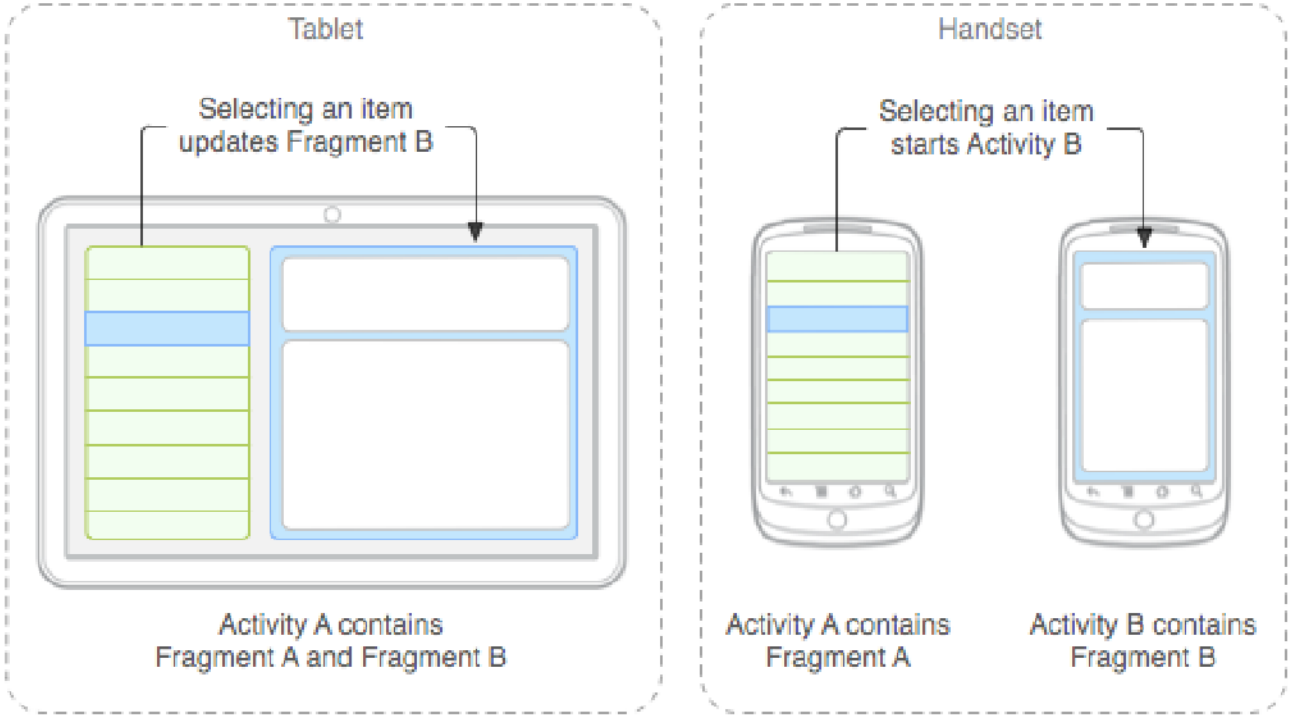
Índice

1. **Fragments**
2. Ciclos de vida
3. Vinculación de Activity con Fragments
4. Administración de Fragments
5. Subclases de Fragments
6. Comunicación entre Activity y Fragment
7. Fragment Dialog

Fragments (I)

- Nacieron con la idea de crear interfaces multipanel
- Representa un comportamiento o una parte de la interfaz de usuario
- Tiene su ciclo de vida propio
  - Puede crearse y destruirse mientras la actividad está en ejecución
- Gestiona sus propios eventos de entrada y modifica el layout
- Una actividad puede tener varios Fragments
- Es algo así como una “subactividad”

Fragments (II)



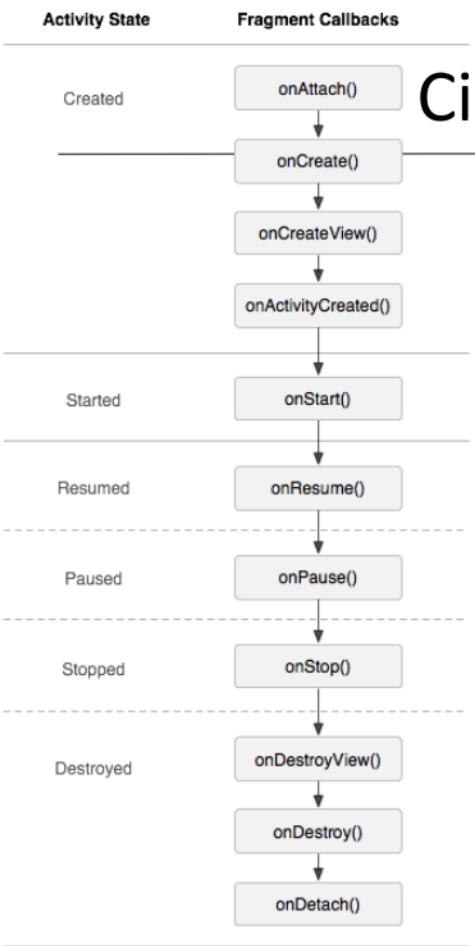
Fragments (y III)

- Se introducen en Android 3.0
  - API 11
- Diseños de IU más dinámicos y flexibles en pantallas grandes
  - Aparición y proliferación de las tabletas
  - Más espacio para combinar e intercambiar componentes de la IU
- Intento de solución al problema de la fragmentación en Android
- Es reusable y puede ser llamada por diferentes actividades (evitar dependencias)

Índice

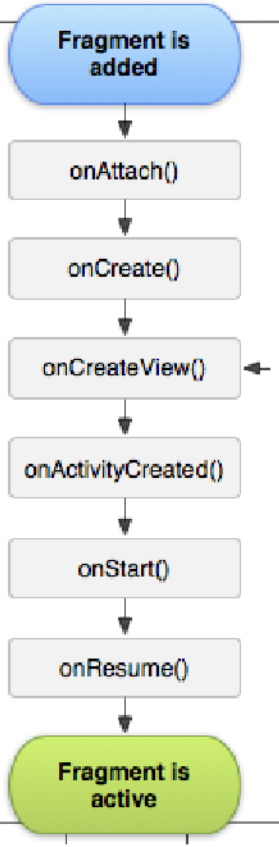
1. Fragments
2. **Ciclos de vida**
3. Vinculación de Activity con Fragments
4. Administración de Fragments
5. Subclases de Fragments
6. Comunicación entre Activity y Fragment
7. Fragment Dialog

Ciclo de vida de Activity y Fragment



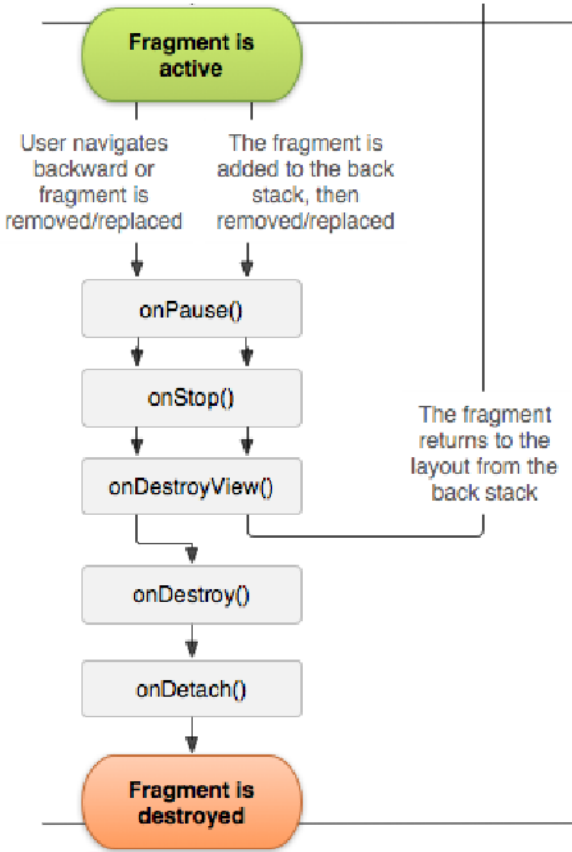
- Como el fragmento está contenido en la actividad, el ciclo de vida de la Activity afecta directamente al del Fragment
  - onPause (Activity) -> onPause (Fragment)
  - onStart (Activity) -> onStart (Fragment)
  - onDestroy (Activity) -> onDestroy (Fragment)
- Pero el Fragment también tiene su propio ciclo de vida

Ciclo de vida (I)



- *onAttach*: primer método que se ejecuta cuando el Fragment se "asocia" a la Activity
- *onCreate*: la Activity lo llama para crear el Fragment
- *onCreateView*: cuando se va a pintar la vista del Fragment
- *onActivityCreated*: una vez el onCreate de la Actividad ha finalizado
- *onStart*: cuando el Fragment es visible para el usuario
- *onResume*: el usuario ya puede interactuar con el Fragment

Ciclo de vida (y II)



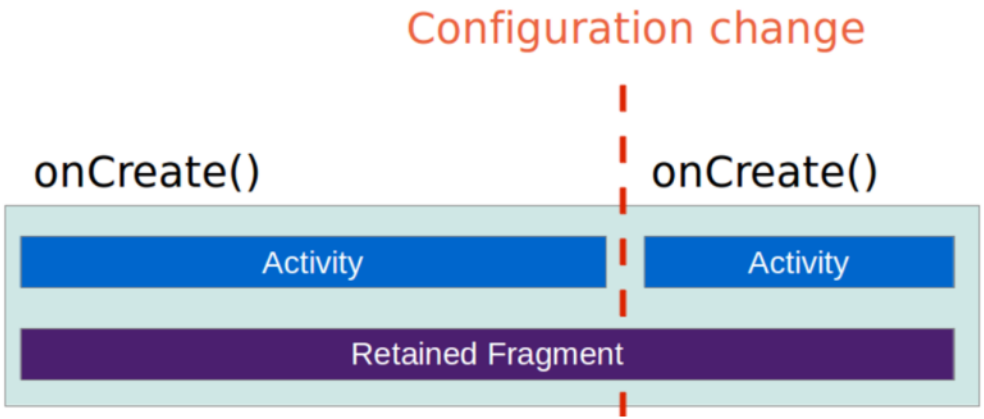
- *onPause*: cuando el usuario deja el Fragment
- *onStop*: cuando el Fragmento ya no es visible
- *onDestroyView*: cuando se desvincula la vista del Fragment
- *onDestroy*: cuando el Fragment ya no está en uso
- *onDetach*: cuando se desvincula el Fragment de la Activity y los recursos se liberan

Cambios de configuración (I)

- Las Actividades se reinician cuando hay cambios de configuración (p. ej.: orientación de la pantalla)
  - onSaveInstanceState(Bundle savedInstanceState)
  - onRestoreInstanceState(Bundle savedInstanceState)
  - Ya explicamos que se puede evitar configurando el AndroidManifest.xml
    - android:configChanges="keyboardHidden|orientation|screenSize"
- En Fragmen puede conservar su configuración
  - Retained Fragment

Cambios de configuración (y II)

- Un Fragment no será destruído aunque lo haga la Activity
- Los métodos del ciclo de vida serán llamados excepto el *onCreate* y el *onDestroy*
- Se activa con el método *setRetainInstance(true)*



Índice

1. Fragments
2. Ciclos de vida
3. **Vinculación de Activity con Fragments**
4. Administración de Fragments
5. Subclases de Fragments
6. Comunicación entre Activity y Fragment
7. Fragment Dialog

Vincular una vista a un Fragment

```
public class ExampleFragment extends Fragment {

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_example, container, false);
    }

}
```

```
@Override
public void onResume() {
    super.onResume();

    textView = getView().findViewById(R.id.textView);
}
```

Vincular una vista a un Fragment (alternativa)

```
public class ExampleFragment extends Fragment {

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        View view = inflater.inflate(R.layout.fragment_example, container, false);

        textView = view.findViewById(R.id.textView);

        return view;
    }
}
```

Añadir un Fragment a una Activity (I)

- Hay dos manera de añadir un Fragment a una Actividad
  - Mediante código (*programmatically*)
  - En el archivo de diseño de la interfaz (en el *layout*)
- Ambas maneras son equivalentes
- Va a depender de:
  - Si los Fragments se cargan de manera estática o dinámica
  - Gustos

Añadir un Fragment a una Activity (II)

- Mediante código (*programmatically*) (el *tag* es opcional, pero recomendable)

```
FragmentManager fragmentManager = getSupportFragmentManager();
FragmentTransaction fragmentTransaction = fragmentManager.beginTransaction();
ExampleFragment fragment = new ExampleFragment();
fragmentTransaction.replace(R.id.cm_content_frame_fl, fragment,"myTag");
fragmentTransaction.commit();
```

- Insertarlo en el archivo de diseño de la interfaz, en el *layout*

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:orientation="horizontal"
   android:layout_width="match_parent"
   android:layout_height="match_parent">
   <FrameLayout
       android:id="@+id/cm_content_frame_fl"
       android:layout_width="match_parent"
       android:layout_height="match_parent" />
</LinearLayout>
```

Añadir un Fragment a una Activity (y III)

- En el fichero de diseño del la interfaz (*layout*)

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:orientation="horizontal"
   android:layout_width="match_parent"
   android:layout_height="match_parent">
   <fragment android:name="com.example.news.ArticleListFragment"
       android:id="@+id/list"
       android:layout_weight="1"
       android:layout_width="0dp"
       android:layout_height="match_parent" />
   <fragment android:name="com.example.news.ArticleReaderFragment"
       android:id="@+id/viewer"
       android:layout_weight="2"
       android:layout_width="0dp"
       android:layout_height="match_parent" />
</LinearLayout>
```

Índice

1. Fragments
2. Ciclos de vida
3. Vinculación de Activity con Fragments
4. **Administración de Fragments**
5. Subclases de Fragments
6. Comunicación entre Activity y Fragment
7. Fragment Dialog

Administración de Fragments

- Se debe usar *FragmentManager*
  - Se obtiene llamando a *getSupportFragmentManager()* desde la Activity
- Transacción: conjunto de cambios que quieres realizar al mismo tiempo
  - Métodos: *add()*, *remove()* y *replace()*.
  - *commit()*: se aplica la transacción a la Activity
  - *addToBackStack()*: cuando se elimine un Fragment, este se detendrá y se reanudará si el usuario retrocede en la navegación
    - Si no se llama, cuando se quite el Fragment éste se destruirá cuando se confirme la transacción y el usuario no podrá regresar a él

Índice

1. Fragments
2. Ciclos de vida
3. Vinculación de Activity con Fragments
4. Administración de Fragments
5. **Subclases de Fragments**
6. Comunicación entre Activity y Fragment
7. Fragment Dialog

Subclases de Fragments

- Existen algunas subclases de Fragment que implementan cierta funcionalidad:
  - **DialogFragment**: muestra un diálogo flotante
  - **ListFragment**: muestra una lista de elementos administrados por un adaptador
    - Ten en cuenta que es preferible utilizar *RecyclerView* en lugar de *ListView*
  - **PreferenceFragmentCompat**: muestra una jerarquía de objetos Preference en forma de lista. Este objeto se usa a fin de crear una pantalla de configuración para tu app

Fragments con parámetros (I)

- Por defecto, Android Studio crea los Fragment con parámetros:

```
// the fragment initialization parameters, e.g. ARG_ITEM_NUMBER
private static final String ARG_PARAM1 = "param1";
private static final String ARG_PARAM2 = "param2";

private String mParam1;
private String mParam2;

public ExampleFragment() { }  // Required empty public constructor

public static ExampleFragment newInstance(String param1, String param2) {
   ExampleFragment fragment = new ExampleFragment();
   Bundle args = new Bundle();
   args.putString(ARG_PARAM1, param1);
   args.putString(ARG_PARAM2, param2);
   fragment.setArguments(args);
   return fragment;
}
```

Fragments con parámetros (y II)

- Para acceder a los parámetros se usa

```
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    if (getArguments() != null) {
        mParam1 = getArguments().getString(ARG_PARAM1);
        mParam2 = getArguments().getString(ARG_PARAM2);
    }
}
```

- Para crear un Fragment con parámetros se usa el constructor

```
ExampleFragment fragment3 = ExampleFragment.newInstance("el 1", "el 2");
```

Índice

1. Fragments
2. Ciclos de vida
3. Vinculación de Activity con Fragments
4. Administración de Fragments
5. Subclases de Fragments
6. **Comunicación entre Activity y Fragment**
7. Fragment Dialog

Comunicación entre Activity y Fragment (I)

- Un Fragment tiene que estar siempre asociado a una Activity
  - Puede acceder a ella a través de *getActivity()*
- De la misma manera, la Activity puede acceder a su(s) Fragment(s) por Id o Tag

```
ExampleFragment fragment = (ExampleFragment) getSupportFragmentManager()
        .findFragmentById(R.id.example_fragment);
```

```
ExampleFragment fragment = (ExampleFragment) getSupportFragmentManager()
        .findFragmentByTag("myTag");
```

```
fragmentTransaction.replace(R.id.cm_content_frame_fl, fragment, "myTag");
```

- Forma elegante de comunicación entre Fragment y Activity
- Es común que para sincronizar Fragments, la Activity ejerce de gestora:
  - Los Fragments de una Activity nunca pueden comunicarse entre sí

Comunicación entre Activity y Fragment (II)

- Definimos una interfaz que será la que exponga qué se puede hacer

```
public class FragmentA extends ListFragment {
    ...
    // Container Activity must implement this interface
    public interface OnArticleSelectedListener {
        public void onArticleSelected(Uri articleUri);
    }
    OnArticleSelectedListener listener;
    @Override
    public void onAttach(Context context) {
        super.onAttach(context);
        try {
            listener = (OnArticleSelectedListener) context;
        } catch (ClassCastException e) {
            throw new ClassCastException(context.toString()
                    + " implement OnArticleSelectedListener");
        }
    }
}
```

Comunicación entre Activity y Fragment (III)

- El Fragment avisa a la Activity de los eventos producidos usando la interfaz:

```
public static class FragmentA extends ListFragment {
    ...
    OnArticleSelectedListener listener;
    ...

    @Override
    public void onListItemClick(ListView l, View v, int position, long id) {
        // Append the clicked item's row ID with the content provider Uri
        Uri noteUri = ContentUris.withAppendedId(ArticleColumns.CONTENT_URI, id);
        // Send the event and Uri to the host activity
        listener.onArticleSelected(noteUri);
    }
    ...
}
```

Comunicación entre Activity y Fragment (y IV)

- La Activity tiene que implementar esa interfaz: *OnArticleSelectedListener* y resolver el evento que genera el Fragment y que éste no puede gestionar

```
public class MainActivity extends AppCompatActivity
                          implements FragmentA.OnArticleSelectedListener {
    ...
    @Override
    public void onArticleSelected(Uri articleUri) {
        // Append the clicked item's row ID with the content provider Uri
    }
    ...
}
```

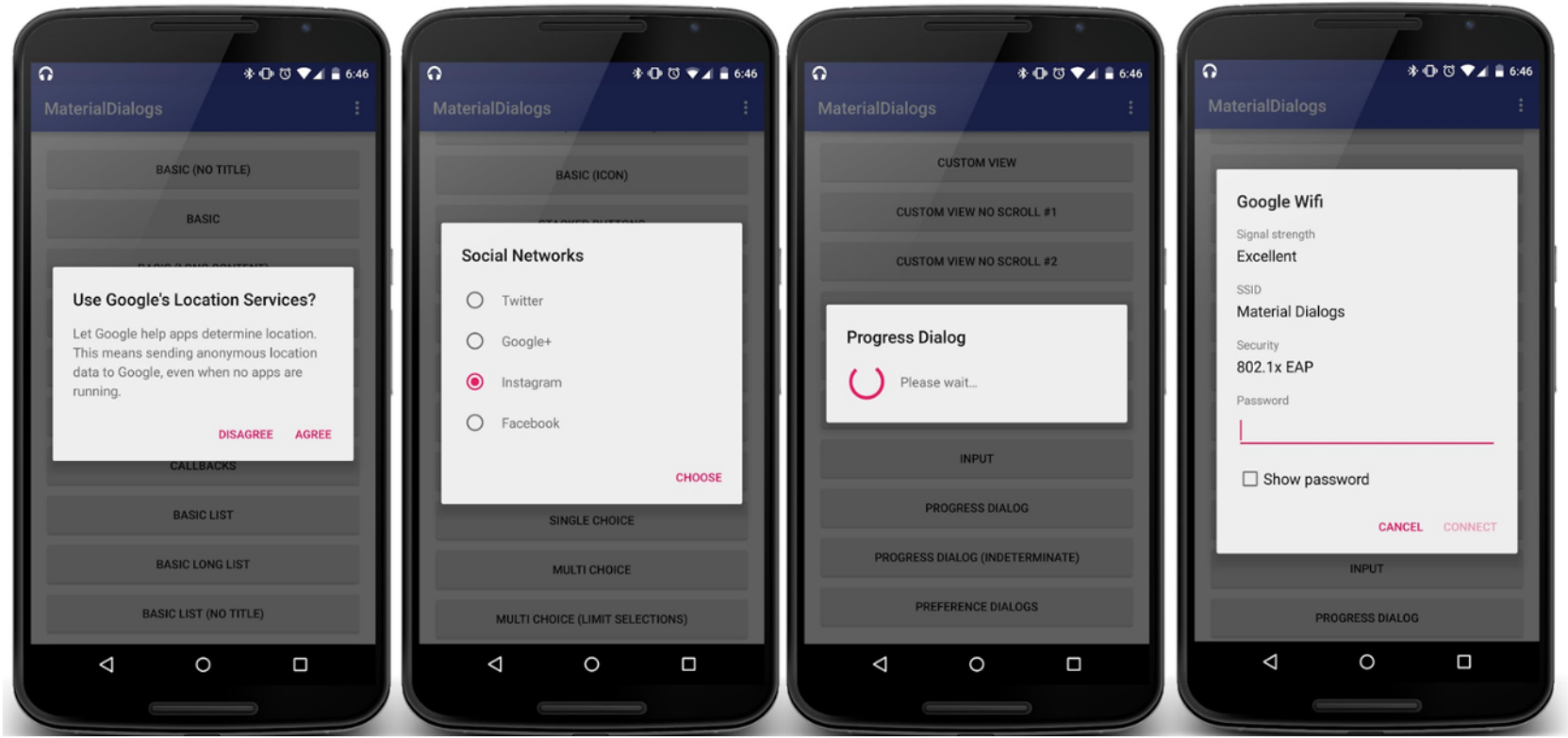
Índice

1. Fragments
2. Ciclos de vida
3. Vinculación de Activity con Fragments
4. Administración de Fragments
5. Subclases de Fragments
6. Comunicación entre Activity y Fragment
7. **Fragment Dialog**

Fragment Dialog (I)

- Pequeñas ventanas que se levantan delante de la Activity
  - Informar y/o pedir información al usuario
  - Al ser Fragment, está vinculada a la Activity
- Se cierra bajo la acción del usuario
  - Botón de aceptar/cerrar
  - Pulsar fuera del diálogo (se puede restringir esto por configuración)
  - Pulsar el botón de atrás
- No se muestran a pantalla completa

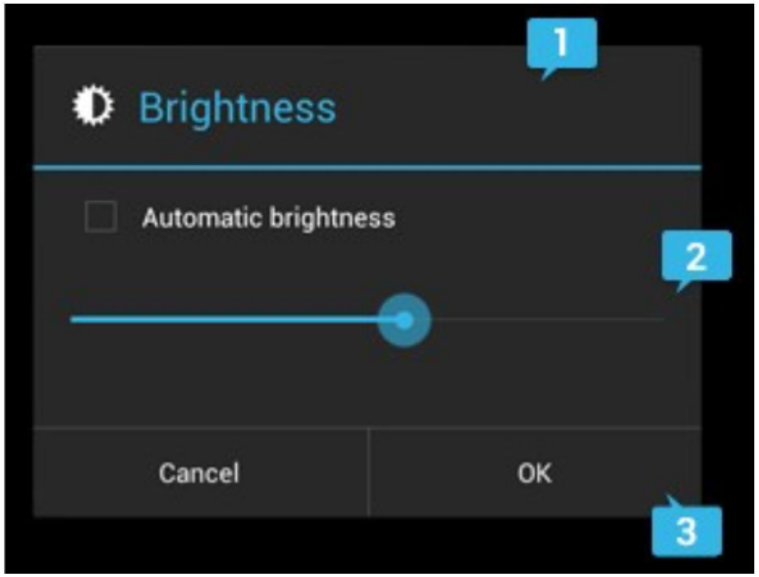
Fragment Dialog (II)



Fuente: https://guides.codepath.com/android/using-dialogfragment

Fragment Dialog (III)

- Varias subclases
  - ProgressDialog
  - DatePickerDialog
  - TimePickerDialog
  - Custom Dialogs
- Estructura
  - Título [1]
  - Contenido [2]
  - Botones de acción [3]
    - Hasta 3 botones



Fragment Dialog (IV)

- Para crear, mostrar y cerrar los diálogos:
  - *new Builder(), create(), show(), dismiss()*
- Añadir los atributos básicos:
  - *setMessage(), setTitle()*
- Añadir los botones y sus *listeners*:
  - *setPositiveButton(), setNegativeButton(), setNeutralButton()*
- Otras opciones
  - *setCancelable(), setIcon()*

Fragment Dialog (y V)

```
final CharSequence[] items = {"Red", "Green", "Blue"};

AlertDialog.Builder builder = new AlertDialog.Builder(this);

builder.setTitle("Pick a color");

builder.setSingleChoiceItems(items, -1, new DialogInterface.OnClickListener() {
   public void onClick(DialogInterface dialog, int item) {
       Toast.makeText(getApplicationContext(), items[item],
Toast.LENGTH_SHORT).show();
   }
});

AlertDialog alert = builder.create();

alert.show();
```

Fragment Dialog con botones

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);

builder.setTitle("Please, confirm action");
builder.setPositiveButton("OK", new DialogInterface.OnClickListener() {
    @Override
    public void onClick(DialogInterface dialogInterface, int i) {
        Toast.makeText(getApplicationContext(),"OK", Toast.LENGTH_SHORT).show();
   }
});
builder.setNegativeButton("Cancel", new DialogInterface.OnClickListener() {
    @Override
    public void onClick(DialogInterface dialogInterface, int i) {
        Toast.makeText(getApplicationContext(),"Cancel", Toast.LENGTH_SHORT).show();
   }
});
AlertDialog alert = builder.create();
alert.show();
```

Fragment Dialog con texto editable

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Write a text");

 final EditText input = new EditText(this);
 LinearLayout.LayoutParams lp = new LinearLayout.LayoutParams(
       LinearLayout.LayoutParams.MATCH_PARENT, LinearLayout.LayoutParams.MATCH_PARENT);
 input.setLayoutParams(lp);
 builder.setView(input);

builder.setPositiveButton("OK", new DialogInterface.OnClickListener() {
    @Override
    public void onClick(DialogInterface dialogInterface, int i) {
        String text = input.getText().toString();
   }
});
AlertDialog alert = builder.create();
alert.show();
```

Referencias

- https://developer.android.com/guide/components/fragments?hl=es-419
- https://guides.codepath.com/android/using-dialogfragment